



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPARTMENT OF ALGORITHMS AND THEIR APPLICATIONS

Design and Implementation of a Two-Player Online Cooperative Game Using the Unity Engine

Supervisor:

Szente Péter

PhD student

Author:

Hassan Ibrahim

Computer Science BSc

Budapest, 2026

Contents

1	Introduction	3
1.1	Problem Background	3
1.2	Motivation for Topic Choice	3
1.3	Thesis Goals	4
1.4	Scope and Constraints	4
2	User Documentation	5
2.1	Short Problem Statement	5
2.2	Brief Method Overview	5
2.3	System Requirements	5
2.4	Installation and Startup	6
2.5	Controls and Interaction	7
2.6	Gameplay Flow	7
2.7	Use Case Diagram	7
2.8	User Stories	7
3	Developer Documentation	11
3.1	Detailed Problem Specification	11
3.2	Concepts and Definitions	12
3.3	System Overview	12
3.4	Logical Structure	13
3.5	Physical Structure	14
3.6	System Modules	14
3.7	Data Structures and State Handling	16
3.8	Class Diagram	17
3.9	Testing Methodology	17
3.10	Test Results	17

4 Conclusion	19
4.1 Summary of Results	19
4.2 Possible Extensions	20
Acknowledgements	21
Bibliography	22
List of Figures	23
List of Tables	24

Chapter 1

Introduction

1.1 Problem Background

Online cooperative puzzle games require effective communication between participants interacting within a shared environment. This thesis presents *A Task for Two*, a two-player online cooperative first-person puzzle system in which both users receive partial information and must collaborate to complete the objective. The system was implemented in Unity using Netcode for GameObjects and Relay-based online session management [1, 2, 3].

The core gameplay mechanic is a synchronized symbol puzzle. Each participant is presented with a different equation and controls a die that influences the counterpart's missing value. The puzzle is solved only when both users set the correct values and maintain the correct state long enough for the gate bars to open. This mechanism enforces progress through communication rather than random trial-and-error interaction.

1.2 Motivation for Topic Choice

The topic was chosen because it combines practical game development with core software engineering challenges, including real-time interaction, network synchronization, and reliable user-facing behavior. A cooperative puzzle system provides a well-defined scope for demonstrating requirements analysis, architectural design, implementation, and testing within the constraints of a bachelor thesis.

1.3 Thesis Goals

The primary goal of the project is to design and implement a robust cooperative system prototype that is technically reliable and maintainable. The software goals are:

- Ensure that puzzle logic is deterministic and consistently synchronized across the network.
- Provide a stable host–client lifecycle for session creation and joining.
- Maintain clear separation of gameplay responsibilities across system components.
- Support configurable user options with persistent storage.
- Verify system behavior through automated and manual testing.

The educational goal of the thesis is to demonstrate full-cycle software engineering: requirements analysis, architecture and implementation decisions, testing, and evaluation.

1.4 Scope and Constraints

The scope of the thesis is a complete and stable cooperative gameplay vertical slice. It includes session creation and joining, synchronized puzzle interaction, level completion, and end-screen flow. The implementation also provides in-game menus, persistent user options, runtime error handling, and behavior-focused automated tests.

The objective is engineering quality in a thesis-sized system prototype while prioritizing reliability and maintainability over feature breadth. Therefore, large-scale production features (for example, global matchmaking at scale, anti-cheat systems, or a multi-level content pipeline) are intentionally out of scope. This keeps the work focused on correctness, maintainability, and reproducible behavior in real two-player sessions.

Chapter 2

User Documentation

2.1 Short Problem Statement

This application provides a cooperative two-player puzzle experience in which each participant has only partial information. The task can only be completed if both users communicate and set the correct puzzle values together.

2.2 Brief Method Overview

The system is based on synchronized state sharing between two connected participants. Each participant sees a different puzzle view and controls one die that influences the other participant's missing value. The puzzle is considered solved only when both dice are set to the correct values and remain stable long enough for the bars to open. This design enforces coordinated interaction instead of random rapid input.

2.3 System Requirements

User Requirements

The application requires a Windows-based system capable of running the Unity standalone build. A keyboard and mouse are required for interaction.

Development Requirements (optional)

For development and running from source, the following tools are required:

- Unity Hub.
- Unity Editor 6000.3.10f1.
- Git.
- Git LFS.
- Visual Studio Code 1.111.0.

Platform note:

- Windows standalone build support is available by default on the Windows Unity installation used for this project.
- Additional build targets can be installed from Unity Hub if needed.

2.4 Installation and Startup

1. The application can be obtained by cloning the project repository:

```
git clone https://github.com/hassan-afifi/A_Task_For_Two.git
```

or

```
git clone git@github.com:hassan-afifi/A_Task_For_Two.git
```

2. Enter the project folder:

```
cd A_Task_For_Two
```

3. Enable and pull Git LFS assets:

```
git lfs install
```

```
git lfs pull
```

4. Open Unity Hub, choose **Add project from disk**, select the cloned folder, and open it with Unity Editor 6000.3.10f1.

2.5 Controls and Interaction

The game uses standard first-person controls with keyboard movement and mouse-based camera control. Interaction with objects, such as the puzzle die, is performed using the designated interact key.

2.6 Gameplay Flow

1. Open the main menu and create or join a session.
2. Move to the puzzle area once both participants are connected.
3. Read your local puzzle symbols and communicate with the other participant.
4. Rotate your die to provide the value required by the other participant's puzzle.
5. Keep both dice on correct values until the bars open.
6. Move forward to complete the level and trigger the end screen.

2.7 Use Case Diagram

2.8 User Stories

The following user stories summarize expected user-facing behavior in Given–When–Then format.

Main Menu and Session Flow

Table 2.1: User Stories: Main Menu and Session Flow

I want to	Given	When	Then
Open the options menu	The main menu is visible	I click <i>Options</i>	The application displays the options screen
Close the options menu	The options screen is visible	I click <i>Back</i> or <i>Close</i>	The system returns to the main menu panel

Create a host session	I entered a valid player name	I click <i>Create Game</i>	The system starts host creation and transitions to gameplay
Join an existing session	I entered a valid player name and join code	I click <i>Join Game</i>	The system starts client connection and joins the host session
Avoid invalid session actions	Name or code input is invalid	I stay on the menu screen	The system keeps create/join actions unavailable
Exit the application	I am in a menu screen	I click <i>Exit</i>	The system closes the application

Puzzle Interaction

Table 2.2: User Stories: Puzzle Interaction

I want to	Given	When	Then
See my local puzzle information	The gameplay scene is active	I approach the puzzle area	The game shows my local symbols and target equation values
Rotate my assigned die	I am close to the die and interaction is available	I press the interact key	The system performs one die roll step
Provide the other player's missing value	Both participants are communicating puzzle data	I set my die to the requested face	The shared puzzle state advances toward a valid solution
Solve the puzzle cooperatively	Both dice are on correct values	The values remain stable for the hold duration	The system marks the puzzle as solved
Open the bars after a valid solve	The puzzle was solved	The solve state is confirmed	The application starts the bars opening animation

Prevent post-solve random changes	The puzzle is solved	I try to rotate a die again	The game keeps dice locked and ignores further rolls
-----------------------------------	----------------------	-----------------------------	--

Pause and Options During Gameplay

Table 2.3: User Stories: Pause and Options During Gameplay

I want to	Given	When	Then
Open the pause menu	Gameplay is active and no blocking screen is shown	I press the pause input	The system displays the pause menu overlay
Resume gameplay	The pause menu is open	I click <i>Continue</i> or close the menu	The application hides the menu and restores control
Open options from pause	The pause menu is open	I click <i>Options</i>	The application displays the options screen
Apply changed options	The options screen is open	I change a setting value	The system applies the setting and stores it persistently
Keep gameplay blocked while menu is open	Pause or options UI is open	I attempt world interaction	The game ignores gameplay interaction input until UI is closed

End and Recovery Flow

Table 2.4: User Stories: End and Recovery Flow

I want to	Given	When	Then
-----------	-------	------	------

Finish the level for both players	The bars are open and at least one player reaches the end trigger	The trigger is entered	The system shows the end screen for all participants
Stop gameplay actions on victory	The end screen is shown	I try to move or interact	The game keeps gameplay controls disabled
Return to main menu after victory	The end screen is shown	I click <i>Main Menu</i>	The application returns to the main menu scene
Handle disconnect safely	A client disconnect occurs before victory	Network callback is received	The system shows the connection-lost screen
Exit from the connection-lost screen	The connection-lost screen is open	I click <i>Exit</i>	The system closes the application

Chapter 3

Developer Documentation

3.1 Detailed Problem Specification

The implemented software is a two-player online cooperative puzzle game prototype. The central technical problem is to keep gameplay state synchronized between two remote players while preserving a clear division of responsibilities between UI, networking, gameplay, and persistence.

The game loop is defined by the following functional requirements:

1. Two players can establish an online session through host creation and join-code-based client connection.
2. Both players load into the same gameplay scene and can control their own player object only.
3. Each player receives a local puzzle view that contains partial information.
4. Each player can rotate exactly one interactive die, and that rotation affects the shared puzzle solution state.
5. The puzzle is solved only if both dice are on the required values and remain correct for a hold duration.
6. When solved, the bars are raised and level completion becomes possible.
7. If one player reaches the end trigger, the end screen is shown for both players.
8. If a network disconnect happens before victory, a connection lost screen is shown.

The non-functional requirements are:

- deterministic puzzle resolution and server-authoritative state updates,
- stable session lifecycle transitions (menu, game, return),
- maintainable script boundaries and explicit setup validation,
- persistent player and options settings between sessions,
- automated behavior tests for critical game and UI paths.

3.2 Concepts and Definitions

The most important terms used in this thesis are:

- **Host**: a client that also runs server authority for gameplay state.
- **Client**: a remote participant connected to the host session.
- **Server-authoritative state**: gameplay state that is computed and committed on server side logic, then replicated [1, 2, 4].
- **NetworkVariable**: a synchronized state container in Netcode for GameObjects [1].
- **RPC**: remote procedure call used to request actions or broadcast events [2].
- **Owner**: the client that has control authority over a specific networked object [4].

3.3 System Overview

The application is implemented in Unity using C# scripts, Unity Netcode for GameObjects for replication and object ownership, and Unity Relay for online session transport [1, 2, 3].

At runtime, the architecture can be summarized in four cooperating layers:

- **Session and networking layer**: session creation, join, scene transition, ownership handling.
- **Gameplay layer**: player movement, interaction, synchronized puzzle logic, and completion flow.

- **UI and settings layer:** menus, options, HUD visibility, and saved user preferences.
- **Verification layer:** edit mode and play mode tests that validate expected behavior.

The runtime flow starts in the menu scene, where local UI validates player input and requests session actions. After session establishment, both participants transition to gameplay with network ownership assigned to their local player objects. During gameplay, authoritative state mutations are committed through networked components, while local UI and presentation components react to replicated values instead of directly mutating shared state.

This design keeps network-critical code isolated from purely local UI code, reduces coupling, and prevents hidden dependencies between menu behavior and multiplayer authority rules.

3.4 Logical Structure

Logically, the project is split into script groups by responsibility:

- **Session and flow:** MainMenu, RelayManager, GameSession, MenuActions, PauseMenu, ConnectionLost, EndScreen.
- **Player control and presentation:** PlayerInputHandler, PlayerMovement, PlayerInteractor, PlayerVisuals, PlayerAudio, CharacterAnimation, CharacterSelection.
- **Puzzle domain:** PuzzleController, DieClickAnimate.
- **Options and widgets:** OptionsMenu, SliderHelper, AnimatedToggle, GenderToggle, DropdownHelper, HudStats, MenuSfx.
- **Cross-cutting services:** AudioManager.

The grouping is intentional: UI scripts focus on interaction and presentation, gameplay scripts focus on rules and state transitions, and networking scripts focus on transport and replication boundaries. This choice ensures that UI components remain decoupled from authority logic, while gameplay systems operate on validated state updates only.

The key logical dependency is that UI requests high-level actions (create game, join game, regenerate puzzle), while authoritative shared state changes are validated and propagated through networked components.

3.5 Physical Structure

The physical project layout follows standard Unity organization:

- **Assets/Scenes**: main menu and gameplay scenes.
- **Assets/Prefabs**: reusable runtime objects (UI elements, player related prefabs, puzzle related prefabs).
- **Assets/Scripts**: game logic grouped by responsibility.
- **Assets/Tests/EditMode**: fast logic and component tests.
- **Assets/Tests/PlayMode**: runtime interaction and networked behavior tests.
- **ProjectSettings**: engine, rendering, input, and quality configuration.

This structure supports clear navigation for development and reproducible builds.

3.6 System Modules

Session and Networking Module

This module manages the host/client session lifecycle from menu input to gameplay transition. **MainMenu** validates user inputs and triggers **RelayManager**. **RelayManager** initializes Unity Services, allocates or joins relay sessions, configures transport, and starts host/client networking. **GameSession** stores session-level values such as player name, selected character, and join code.

The main design decision is to keep transport/session orchestration outside gameplay scripts. This prevents gameplay logic from depending on low-level relay initialization details and makes session failures easier to isolate and recover from.

Player Module

The player module splits responsibilities into input, movement, interaction, visuals, and audio. **PlayerInputHandler** captures local actions. **PlayerMovement** applies local first-person control and camera behavior. **PlayerInteractor** performs die interaction raycasts. **PlayerVisuals** replicates character appearance and name. **PlayerAudio** replicates movement/audio cues.

This design keeps local responsiveness high while preserving network consistency. Input is processed locally for responsiveness, while shared presentation and interaction outcomes are synchronized through network-aware components.

Puzzle Module

The puzzle module is centered on `PuzzleController` and `DieClickAnimate`. Dice rolls are synchronized and lockable. Puzzle equations are generated and distributed so each side solves the other side's missing value. Solve state is committed only when both dice stay correct for the hold duration.

Dice state is synchronized with network variables to guarantee that both clients observe the same faces and lock state. The lock mechanism is required to prevent late input or race-like updates from invalidating a valid solve after confirmation. The hold duration requirement enforces intentional coordination and reduces accidental success caused by rapid random rotations.

UI and Options Module

`OptionsMenu` manages display, audio, control, and HUD preferences with persistent storage in `PlayerPrefs` [5]. `SliderHelper`, `AnimatedToggle`, `GenderToggle`, and `DropdownHelper` provide reusable UI behavior components.

UI modules are intentionally reusable and mostly independent from session authority. This reduces duplication between main menu and pause menu contexts and allows the same setting logic to be applied consistently in both scenes.

End and Recovery Module

`EndScreen` broadcasts victory view state to all participants and disables gameplay controls locally. `ConnectionLost` listens for disconnect callbacks and displays a recovery screen when sessions end unexpectedly.

This distinction separates successful completion flow from failure recovery flow. As a result, disconnect handling remains explicit and does not interfere with victory-state transitions.

3.7 Data Structures and State Handling

The project does not use an external relational database. State is handled through a combination of runtime in-memory objects, synchronized network structures, and key-value local persistence.

Runtime and Networked State

`PuzzleController` defines dedicated structs for puzzle state:

- `PuzzleEquation`: missing value, visible values, and sum.
- `NetworkPuzzleState`: both equations, two targets, solved flag, and version.

These values are synchronized using `NetworkVariable<NetworkPuzzleState>` and netcode serialization. `DieClickAnimate` similarly synchronizes active face index and lock state with network variables.

This representation was chosen to keep the replicated state compact and deterministic. Bundling puzzle values into well-defined structs minimizes scattered synchronization points and makes versioned puzzle regeneration simpler to reason about.

Persistent Local State

Longer-lived user preferences are stored in `PlayerPrefs` [5]. Important persisted groups are:

- player identity data (name, selected character),
- visual settings (display mode, resolution, quality, FOV, crosshair),
- audio settings (master and subgroup volumes),
- HUD visibility flags (FPS, ping, clock).

This strategy is sufficient for a two-player prototype and avoids unnecessary database complexity.

The combination of network variables for shared runtime state and `PlayerPrefs` for local preferences provides a clear boundary between synchronized gameplay data and device-local configuration.

3.8 Class Diagram

3.9 Testing Methodology

Testing is based on Unity Test Framework in two modes [6]:

- **Edit mode tests:** validate logic and component behavior without full runtime scene execution.
- **Play mode tests:** validate runtime behavior, scene interactions, networking-related behavior, and end-to-end gameplay transitions.

Edit mode tests are used where fast feedback and isolation are most valuable, because they run without full scene bootstrap and can validate logic-level behavior quickly. Play mode tests are used where runtime orchestration matters, such as scene transitions, object lifecycles, and multiplayer interaction timing.

The test design strategy is behavior oriented. Each tested public method has a dedicated test method that verifies expected branches and outcomes for representative input combinations. Tests are grouped into focused files with consistent naming conventions.

The test suite also includes setup and teardown routines to isolate test state (for example, cleanup of transient game objects or network-related singletons between runs).

3.10 Test Results

At the current thesis development snapshot, the automated suite contains **90 tests**:

- **30 edit mode tests** across three edit mode test files,
- **60 play mode tests** across six play mode test files.

The tests cover the most critical behavior paths:

- session creation and join flow,
- menu and pause interactions,
- option persistence and runtime application,
- player spawn/control synchronization boundaries,

- puzzle solve/lock/regenerate behavior,
- connection lost and end-screen transitions.

In addition to automated tests, manual multi-client validation was performed to confirm correct behavior under real host/client conditions and full gameplay progression from session creation to end screen.

Chapter 4

Conclusion

4.1 Summary of Results

This thesis presented the design and implementation of *A Task for Two*, a two-player online cooperative puzzle game prototype developed in Unity. The implemented system delivers a complete gameplay vertical slice: session creation and joining through Relay, synchronized puzzle interaction, level completion logic, and shared end-screen flow.

From an engineering perspective, the project achieved the main technical goals defined in the introduction. Puzzle state is synchronized with server-authoritative updates, player ownership boundaries are enforced for interaction and movement, and menu-to-game lifecycle transitions are handled in a stable way. The codebase is organized into clear modules (session/networking, player, puzzle, UI/options, and end/recovery), which improved maintainability and testability.

User-facing reliability was supported by persistent settings and structured UI flows. Automated verification was implemented with Unity Test Framework in both edit mode and play mode, covering the critical functional paths of session handling, options behavior, player/network interactions, puzzle resolution, and end-state transitions. In addition to automated tests, manual multi-client runs were conducted to validate real-world host/client behavior and full gameplay flow.

Overall, the thesis objective was met: a technically robust and maintainable cooperative game prototype was designed, implemented, and validated within BSc thesis scope constraints.

4.2 Possible Extensions

The current implementation is intentionally focused on a reliable two-player core experience. Future extensions can be grouped by how much architectural change they require:

- **Additional puzzle variants and multi-level progression (incremental extension):** the current puzzle and scene structure already supports adding new puzzle rules and extra levels without redesigning the core networking model.
- **Account-based profiles (requires architectural redesign):** the current implementation stores identity and settings locally, so cross-session account identity requires backend services and server-side profile management.
- **Achievement systems and unlockable rewards (partial extension):** local achievements can be added incrementally, while account-synced progression requires the same account/backend redesign.
- **Support for more than two players (requires major redesign):** core puzzle state, targeting logic, and session assumptions are currently two-player oriented and must be generalized.
- **Public/private rooms with room search (requires session-layer redesign):** the current join-code flow has no room discovery, so lobby/discovery services and related UI flows must be introduced.
- **Voice chat and text chat (requires subsystem integration):** communication features need dedicated transport/services, UI integration, and moderation-related handling.
- **Stronger security and anti-cheat (partial extension):** server-authoritative validation exists for key puzzle actions, but broader anti-cheat hardening requires expanded server-side validation and abuse-resistant network checks.

These extensions are not required for the thesis objective, but they outline a clear and technically grounded path from a prototype implementation toward a more complete and scalable multiplayer product.

The results demonstrate that the proposed architecture and implementation approach are suitable for small-scale cooperative multiplayer games and provide a solid foundation for further development.

Acknowledgements

I would like to thank my supervisor, Szenté Péter, for his guidance and support throughout this project. I would also like to thank my family and friends for their continuous encouragement.

Bibliography

- [1] Unity Technologies. *Netcode for GameObjects: NetworkVariable*. 2026. URL: <https://docs.unity3d.com/Packages/com.unity.netcode.gameobjects@2.5/manual/basics/networkvariable.html>.
- [2] Unity Technologies. *Netcode for GameObjects: RPC Message System*. 2026. URL: <https://docs.unity3d.com/Packages/com.unity.netcode.gameobjects@2.5/manual/advanced-topics/message-system/rpc.html>.
- [3] Unity Technologies. *Unity Relay Documentation*. 2026. URL: <https://docs.unity.com/en-us/relay>.
- [4] Unity Technologies. *Netcode for GameObjects: Ownership*. 2026. URL: <https://docs.unity3d.com/Packages/com.unity.netcode.gameobjects@2.5/manual/basics/ownership.html>.
- [5] Unity Technologies. *Unity Scripting API: PlayerPrefs*. 2026. URL: <https://docs.unity3d.com/ScriptReference/PlayerPrefs.html>.
- [6] Unity Technologies. *Unity Test Framework Manual*. 2021. URL: <https://docs.unity3d.com/Packages/com.unity.test-framework@1.1/manual/index.html>.

List of Figures

List of Tables

2.1	User Stories: Main Menu and Session Flow	7
2.2	User Stories: Puzzle Interaction	8
2.3	User Stories: Pause and Options During Gameplay	9
2.4	User Stories: End and Recovery Flow	9